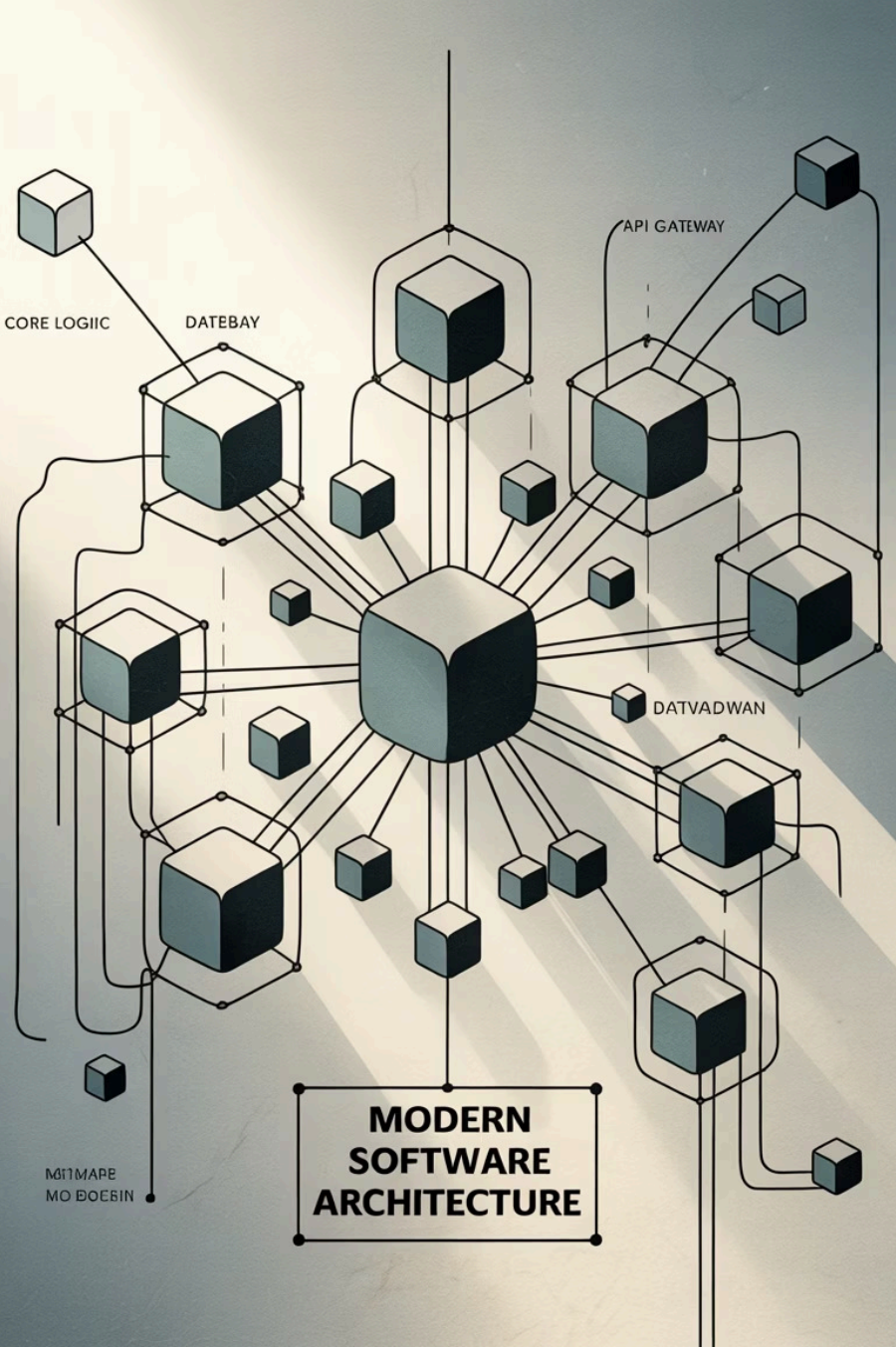




Introduction to Microservices

Welcome to this comprehensive overview of microservices architecture – the approach that has revolutionised how we build and scale modern software applications. This architectural paradigm powers the technological infrastructure of leading companies like Netflix and Amazon, enabling unprecedented levels of agility and scalability in development processes.

Throughout this presentation, we'll explore what makes microservices unique, how they compare to traditional approaches, and why they've become so crucial for organisations navigating today's digital landscape.

by Naresh Chaurasia

What Are Microservices?

Microservices represent an architectural style where applications are composed of small, independent services that each focus on a specific business function. Rather than building one massive application, we create a constellation of specialised services that work together.

Each service:

- Operates independently with its own codebase
- Can be deployed and scaled separately
- Communicates with other services via well-defined APIs
- Typically manages its own data storage



Microservices vs Monolithic Architecture



Monolithic Architecture

Single, unified codebase where all components are tightly integrated

- Shared resources and dependencies
- Entire application must be deployed together
- Changes to one part may affect the whole system
- Simple to develop initially but complex to maintain



Microservices Architecture

Collection of independent, specialised services working together

- Isolated codebases with clear boundaries
- Services can be deployed independently
- Changes to one service don't affect others
- More complex initially but easier to evolve

Key Components of Microservices Architecture

1

Microservices

Small, autonomous services that work together, each focused on a single capability or business domain.

- Built around business capabilities
- Loosely coupled with other services
- Independently deployable

2

API Gateway

Single entry point for all client requests, handling routing, authentication, and load balancing.

- Simplifies client interactions
- Provides centralised security
- Reduces client-side complexity

3

Service Registry & Discovery

Tracks service locations dynamically as they scale up, down, or move.

- Enables dynamic service location
- Supports automatic load balancing
- Facilitates elastic scaling

4

Database per Service

Each microservice manages its own data storage to maintain independence.

- Ensures data autonomy
- Allows technology diversity
- Prevents data coupling

Core Technology Enablers



Containerisation

Docker packages services with all dependencies, ensuring consistent operation across environments. Containers provide isolation, resource efficiency, and rapid deployment capabilities.



Orchestration

Kubernetes automates deployment, scaling, and management of containerised services. It handles service discovery, load balancing, self-healing, and configuration management.



Message Brokers

Tools like Kafka and RabbitMQ enable asynchronous communication between services. They decouple services and support event-driven architectures and resilient interactions.



Advantages of Microservices

Enhanced Scalability

Services can be scaled independently based on demand, optimising resource usage and improving cost efficiency. High-traffic components can be scaled without affecting others.

Accelerated Development

Smaller codebases enable faster development cycles, more frequent releases, and quicker time-to-market for new features. Teams can work in parallel with reduced coordination overhead.

Improved Resilience

Service isolation prevents cascading failures – if one service fails, others continue functioning. This fault isolation creates more robust applications with higher availability.

Technology Diversity

Different services can use different programming languages, frameworks, and databases, allowing teams to select the best tools for each specific task and business requirement.

Main Challenges of Microservices



- Operational Complexity

Managing dozens or hundreds of services requires sophisticated monitoring, deployment, and orchestration tools. Operational overhead is significantly higher than with monoliths.

- Distributed System Challenges

Testing and debugging are more difficult when requests span multiple services. Tracing issues across service boundaries requires specialised tooling and expertise.

- Data Consistency Issues

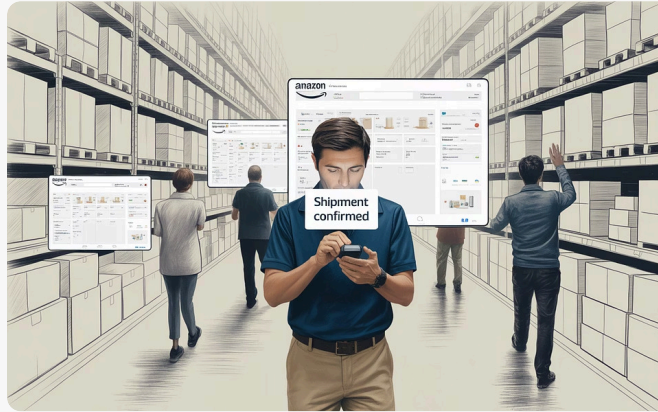
Maintaining data consistency across services is challenging. Distributed transactions are complex, often requiring eventual consistency patterns and careful design.

Real-World Examples



Netflix

Operates over 1,000 microservices that power their global streaming platform. This architecture enabled Netflix to scale to 200+ million subscribers while maintaining 99.99% uptime and rapid feature deployment.



Amazon

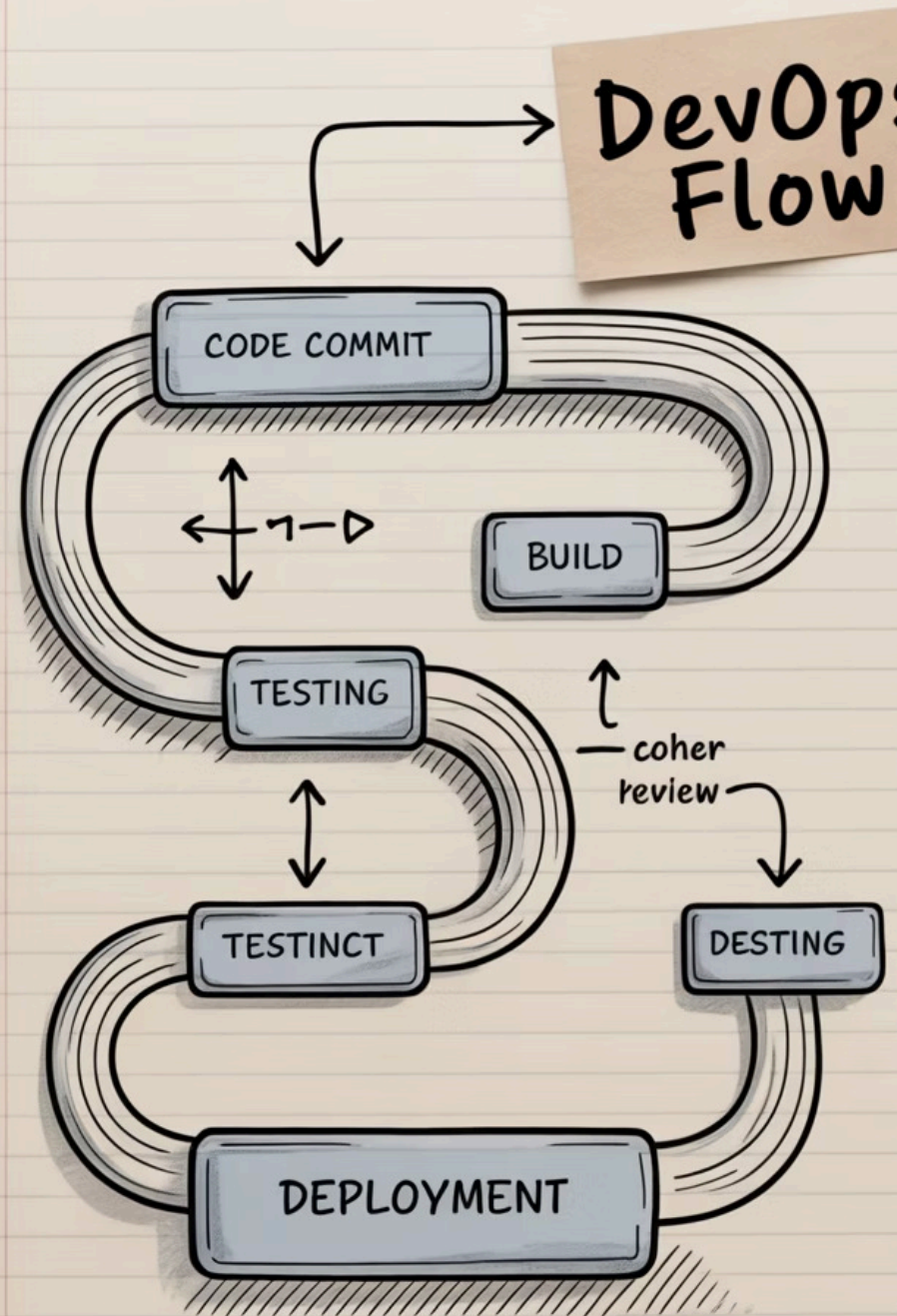
Transformed from a monolithic architecture to microservices in the early 2000s. This shift allowed Amazon to scale its retail platform to handle millions of transactions and evolve rapidly into new business areas.



Spotify

Leverages microservices to enable independent feature teams to develop and deploy autonomously. This architecture supports personalised music recommendations and seamless streaming for 400+ million users.

Best Practices for Microservices



Service Design Principles

- Create small services with single responsibility
- Define clear service boundaries based on business domains
- Design robust APIs with versioning
- Implement proper error handling and graceful degradation

Automation & DevOps

- Implement CI/CD pipelines for continuous delivery
- Automate testing at all levels (unit, integration, contract)
- Use infrastructure as code for consistent environments
- Implement blue-green deployments for zero downtime

Monitoring & Resilience

- Implement comprehensive observability (metrics, logs, traces)
- Track request flows across service boundaries
- Use circuit breakers to prevent cascading failures
- Implement retries, timeouts, and bulkheads for resilience



Conclusion & Next Steps

Key Takeaways

- Microservices fundamentally change how modern applications are built and operated
- They enable unprecedented scalability, resilience, and team autonomy
- The architecture introduces complexity that must be managed with proper tooling and practices
- Leading tech companies have demonstrated their effectiveness at massive scale

Recommended Next Steps

1

Explore Design Patterns

Study microservices patterns like CQRS, Event Sourcing, and API Gateway patterns

2

Learn Migration Strategies

Understand "strangler fig" and other patterns for transitioning monoliths

3

Complete Hands-on Labs

Build a simple microservices system using Docker and Kubernetes